Machine Program: Basics

Jinyang Li

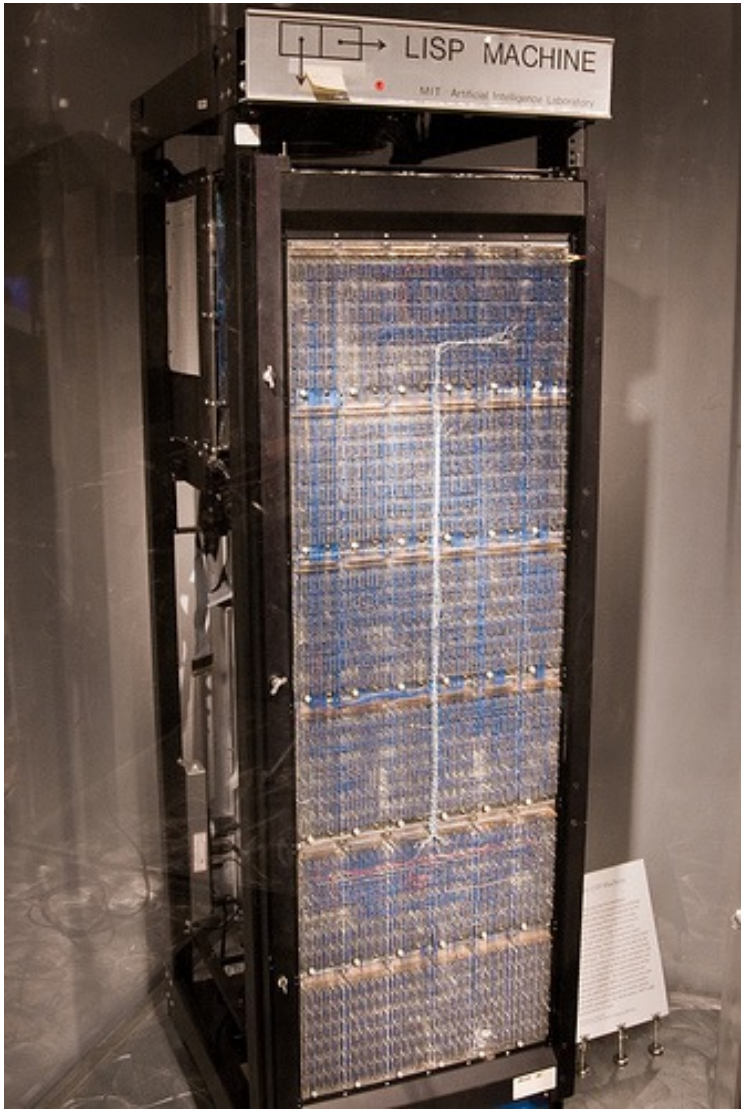
Some are based on Tiger Wang's slides

Lesson plan

- What we've learnt so far:
  - How integers/reals/characters are represented by computers
  - C programming

- Today:
  - Basic hardware execution of a program
  - x86 registers
  - x86 move instruction

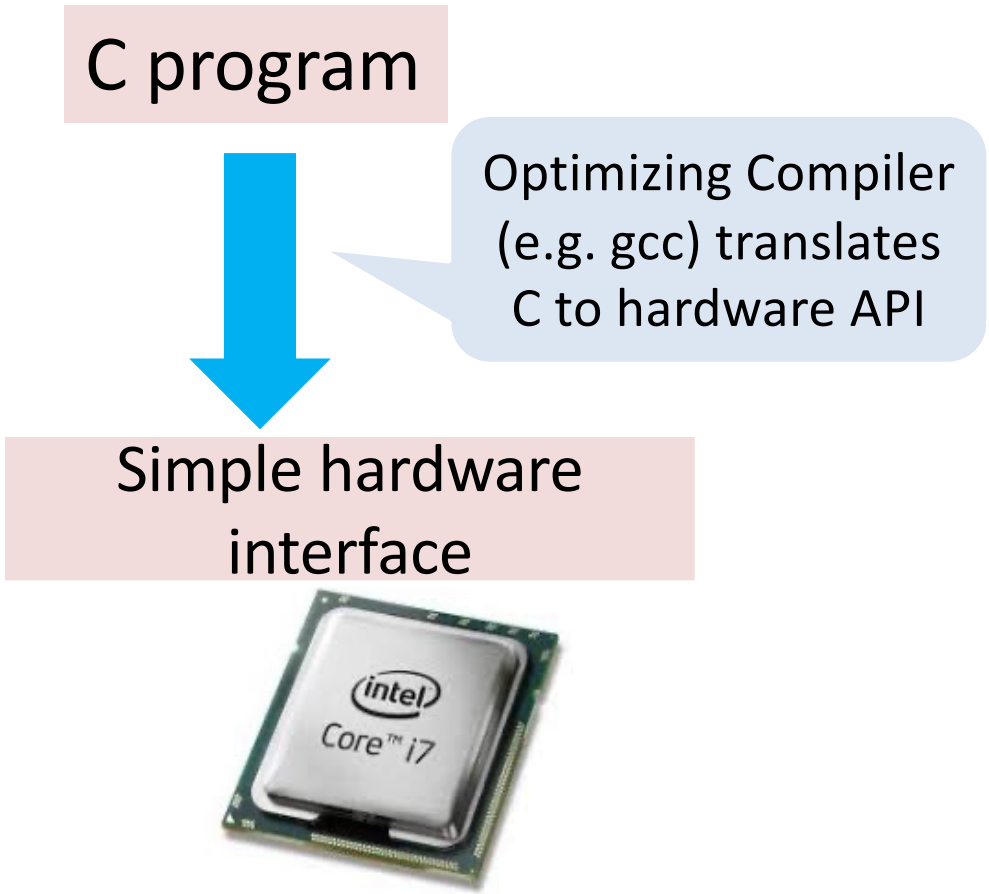
Can we build a machine to execute C directly?



- Historical precedents:
  - LISP machine (80s)
  - Intel iAPX 432 (Ada)

Why not directly execute C?

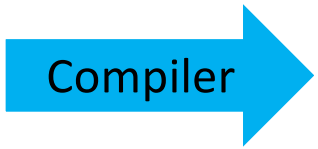
- Results in very complex hardware design
  - Complex → Hard to implement w/ high performance
- A better approach:



C vs. assembly vs. machine code

C source

```
long x;
long y;

y = x;
y = 2*y;
```



x86 assembly

```
movq %rdi, %rax
addq %rax, %rax
```

x86 machine code

```
0100001000001110
1000100101010110
….
```

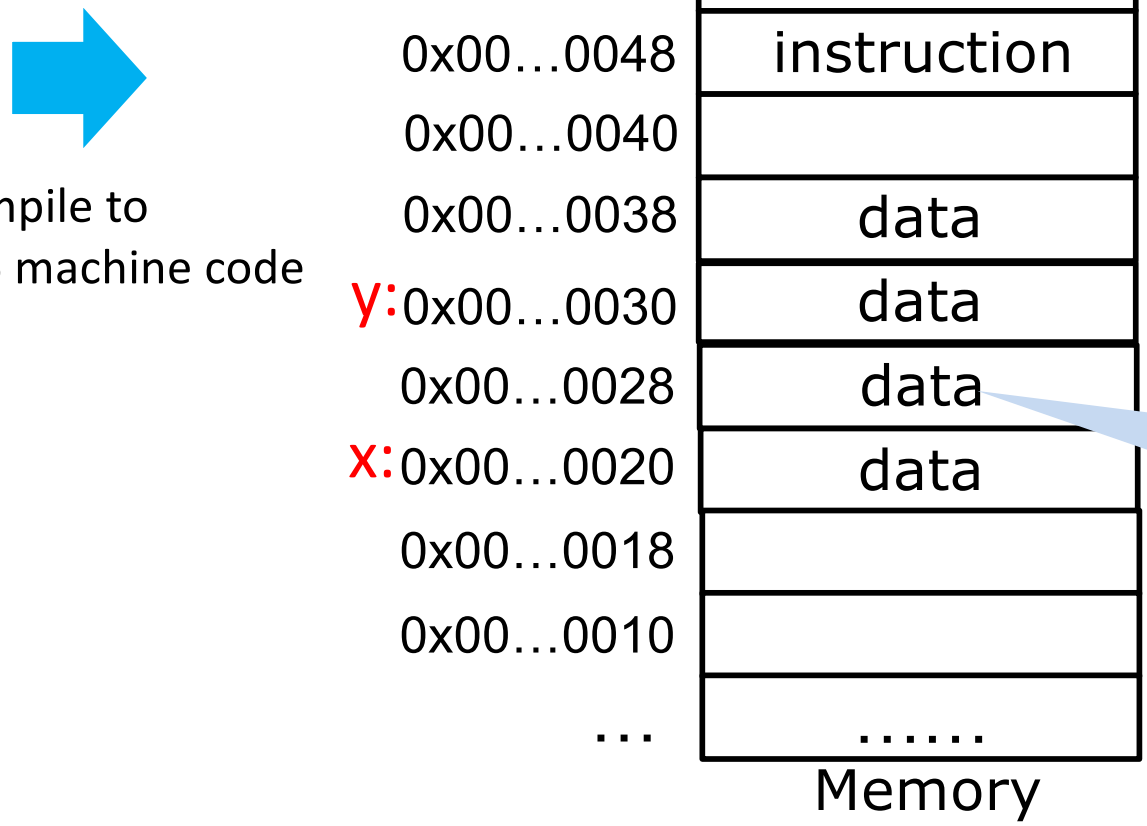
gcc –c does both

gcc –S compiles to assembly

C vs. machine code

```
long x;
long y;

y = x;
y = 2*y;
```

compile to
x86 machine code

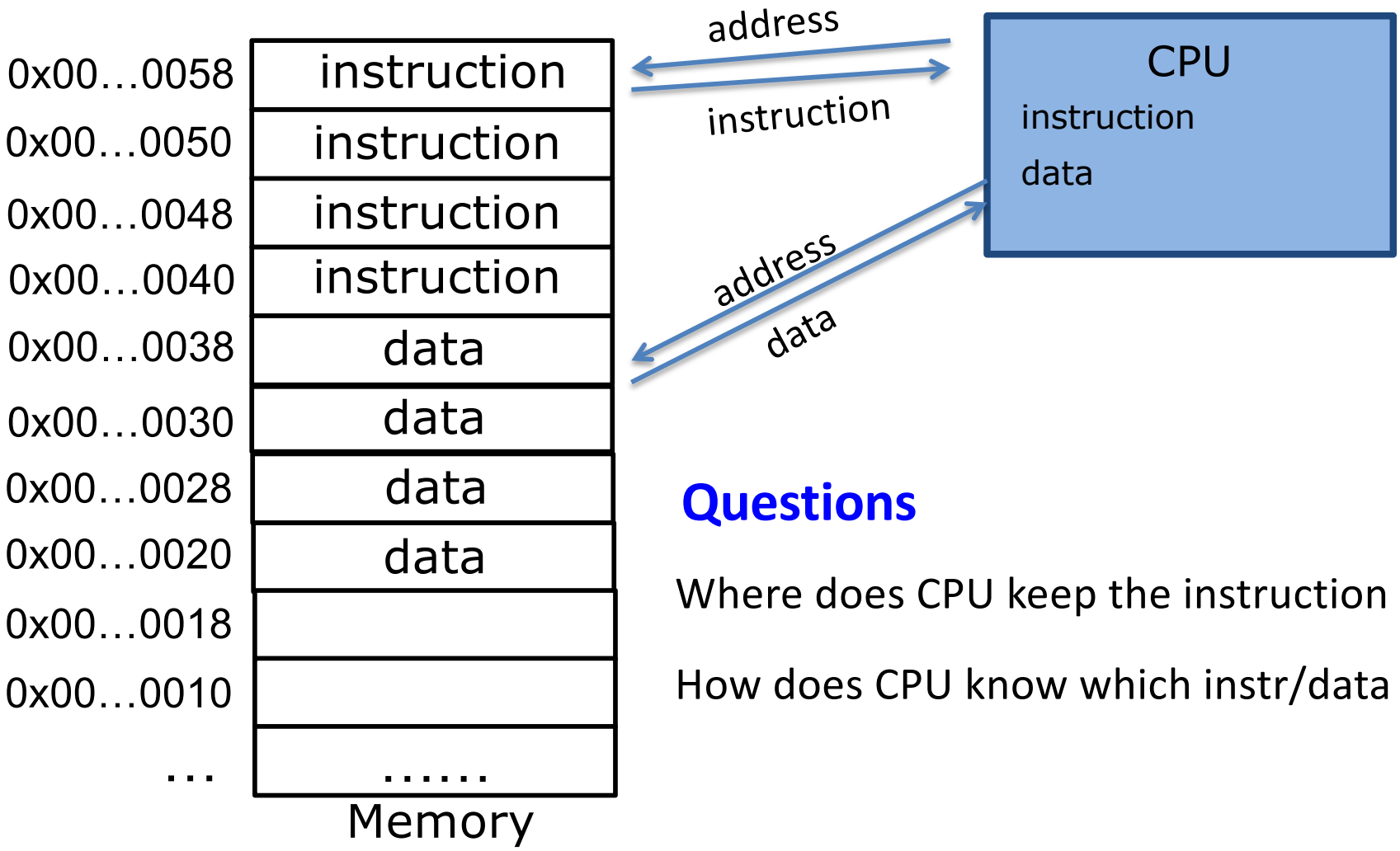


E.g. move data from one memory location to another

E.g. multiply the number at some memory location by a constant

No concept of variables, scopes, types

How CPU executes a program

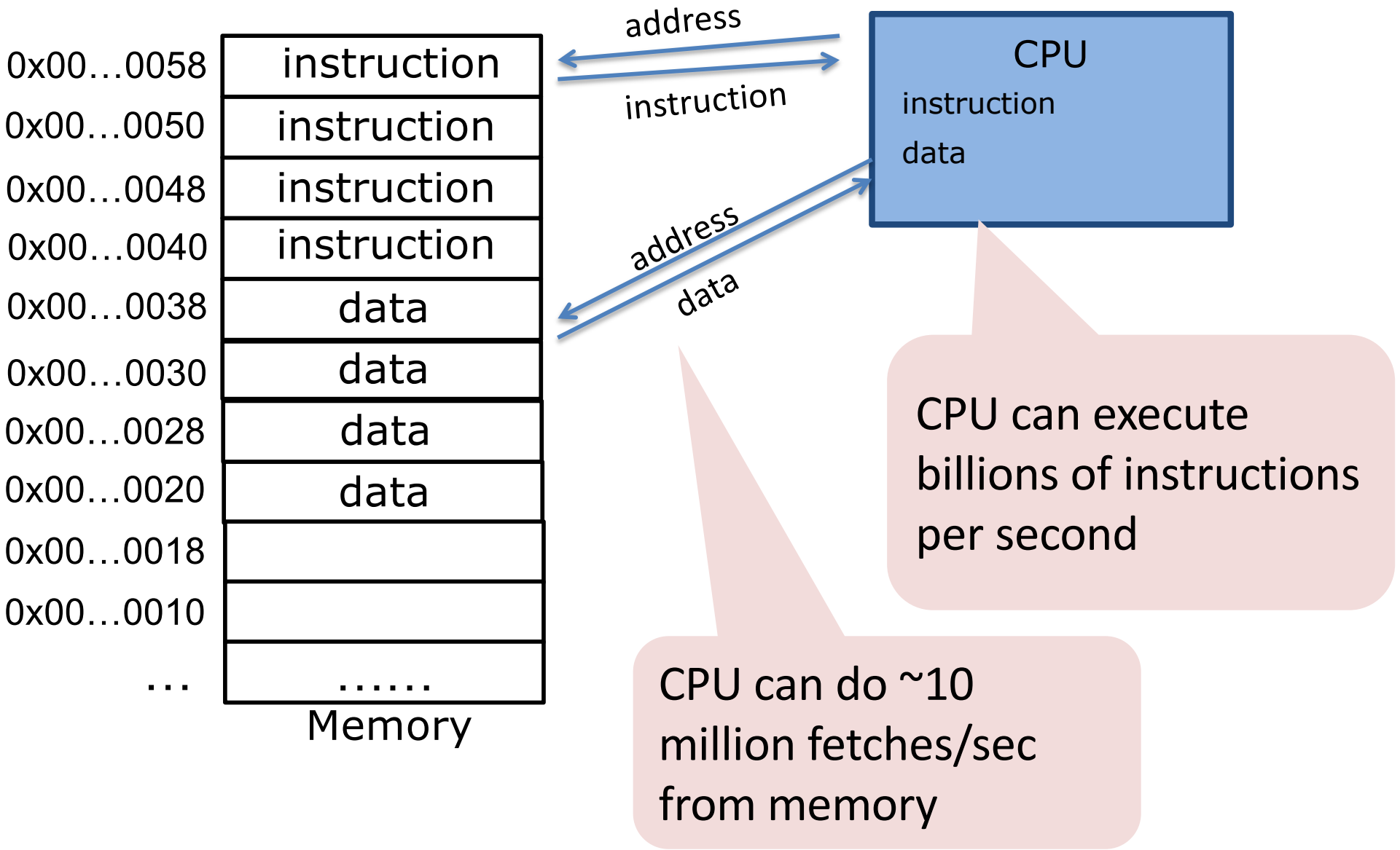


Questions

Where does CPU keep the instruction and data?

How does CPU know which instr/data to fetch?

How CPU executes a program



Register – temporary storage area built into a CPU

PC: Program counter, also called instruction pointer (IP).

- Store memory address of next instruction

IR: CPU's internal buffer storing the fetched instruction

General purpose registers:

- Store data and address used by programs

Program status and control register:

- Status of the instruction executed

Steps of execution in CPU

1. PC contains the instruction's address

2. Fetch the instruction to internal buffer

3. Execute the instruction which does one of following:
   - Memory operations: move data from memory to register (or opposite)
   - Arithmetic operations: add, shift etc.
   - Control flow operations.

4. PC is updated to contain the next instruction's address.

Instruction Set Architecture (ISA)

- ISA: interface exposed by hardware to software writers

- X86_64 is the ISA implemented by Intel/AMD CPUs
  - 64-bit version of x86

Lectures on assembly

- ARM is another common ISA
  - Phones, tablets, Raspberry Pi, Apple's new M1 laptop

- RISC-V is yet another ISA
  - P&H textbook's ISA.
  - Open-sourced, royalty-free

Lectures on hardware

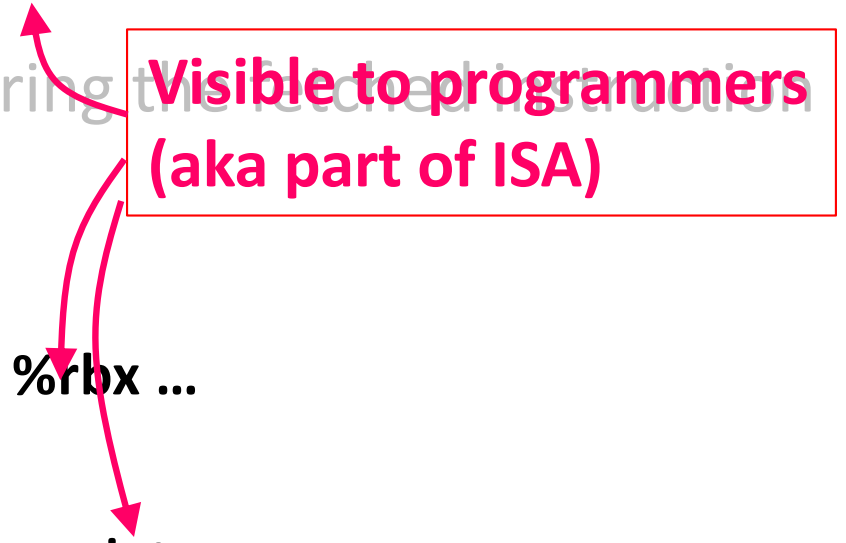
Apple laptops use ARM chips. Hence we need a virtual machine to run C programs compiled to x86

X86-64 ISA: registers

Program counter:

- **called %rip in x86_64**

IR: CPU's internal buffer storing the fetched instruction



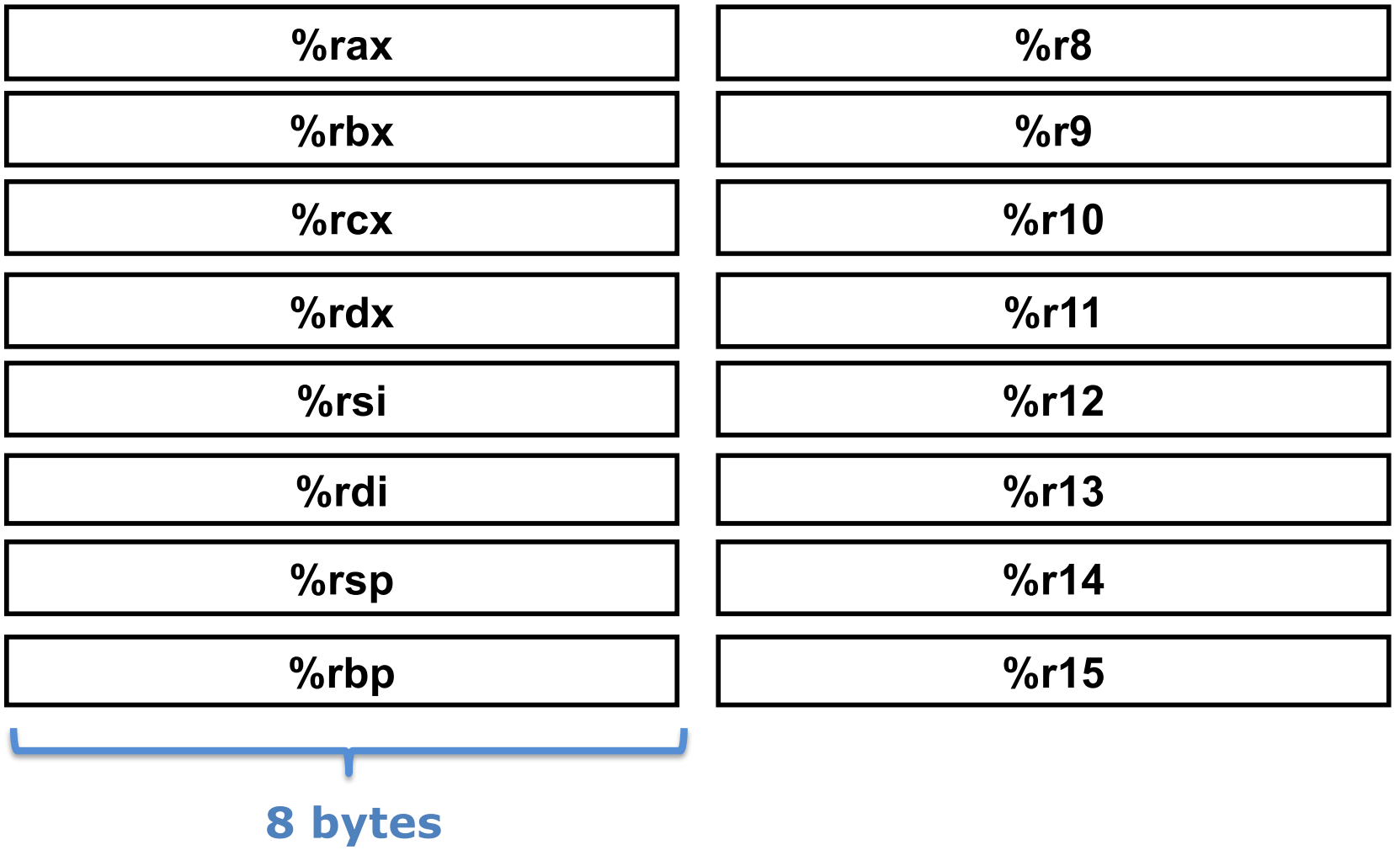
General purpose registers:

- **16 8-byte registers: %rax, %rbx …**

Program status and control register:

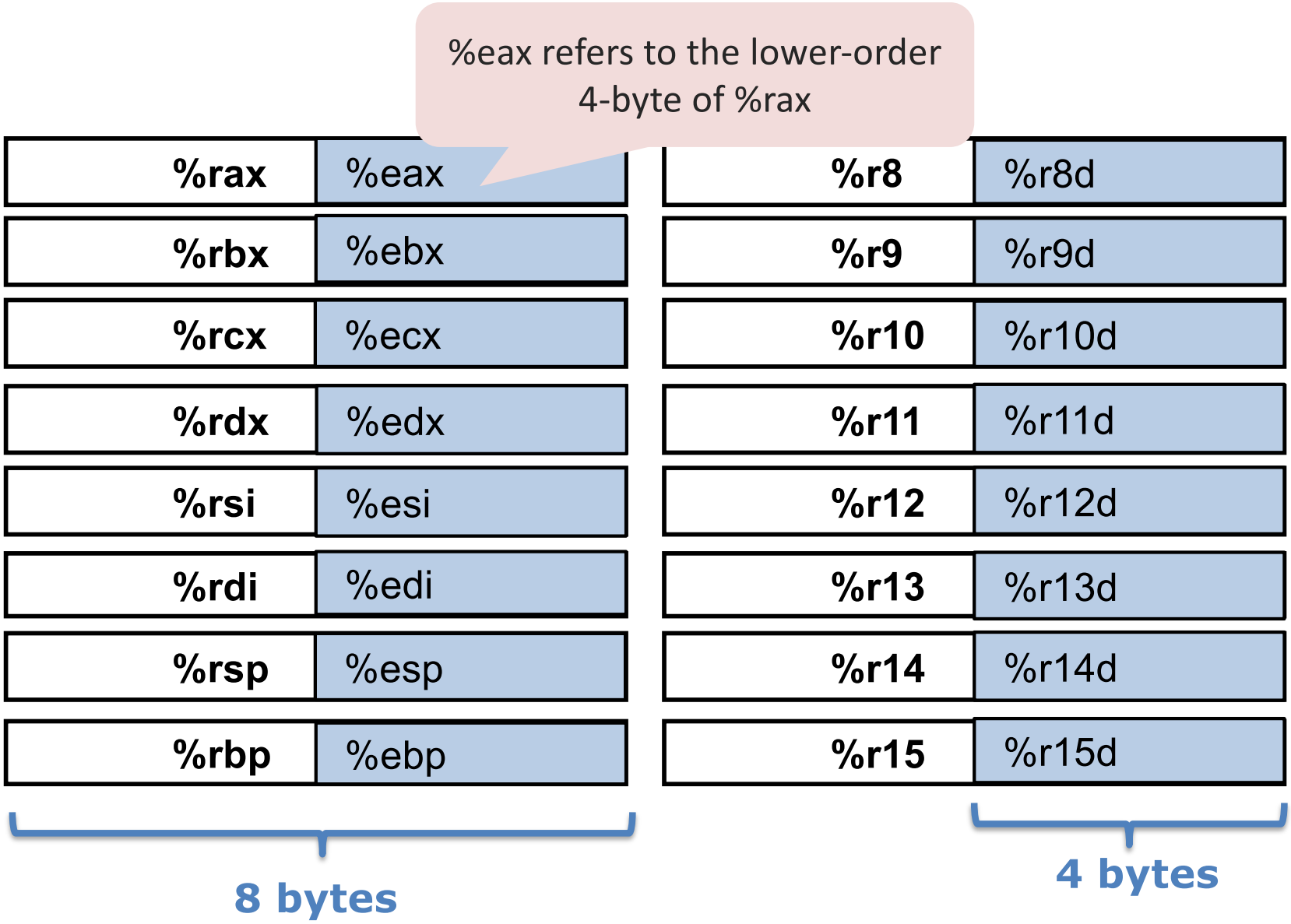
- **Called "RFLAGS" in x86_64**

X86-64 general purpose registers: 8-byte



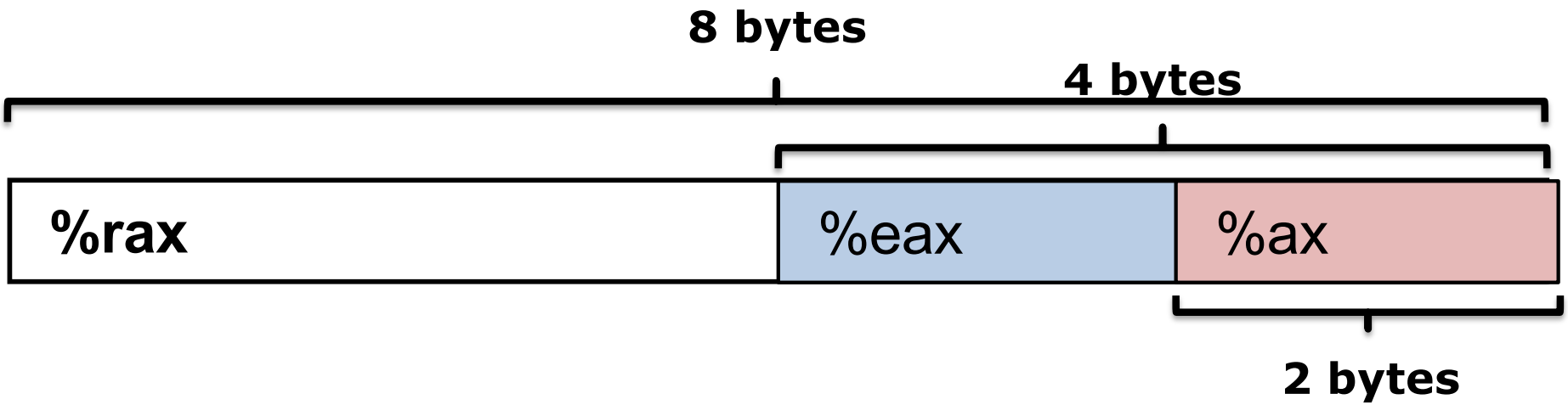
X86-64 general purpose registers: 4-byte

4-byte registers refer to the lower-order 4-bytes of original registers.

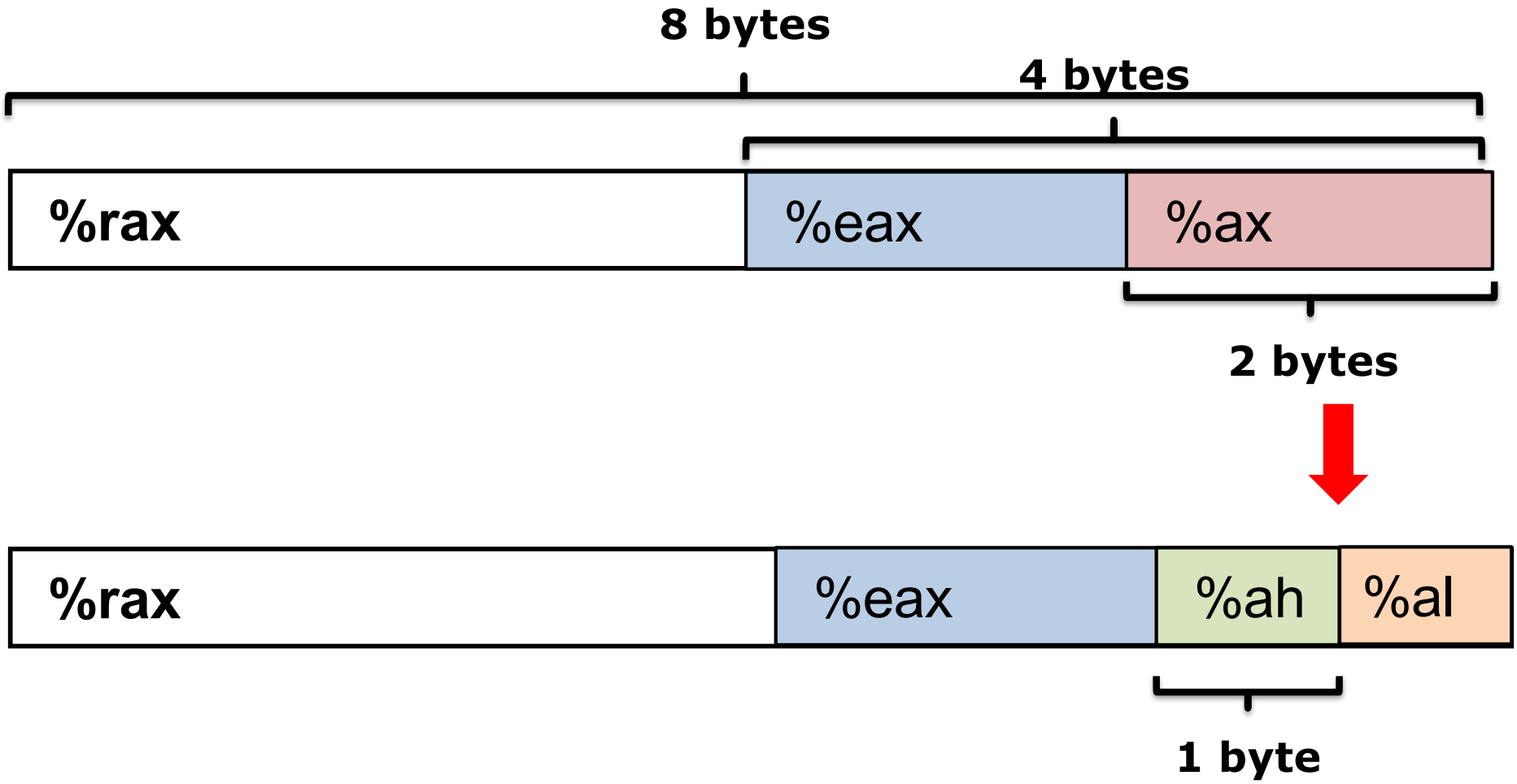


X86-64 general purpose registers: 2-byte

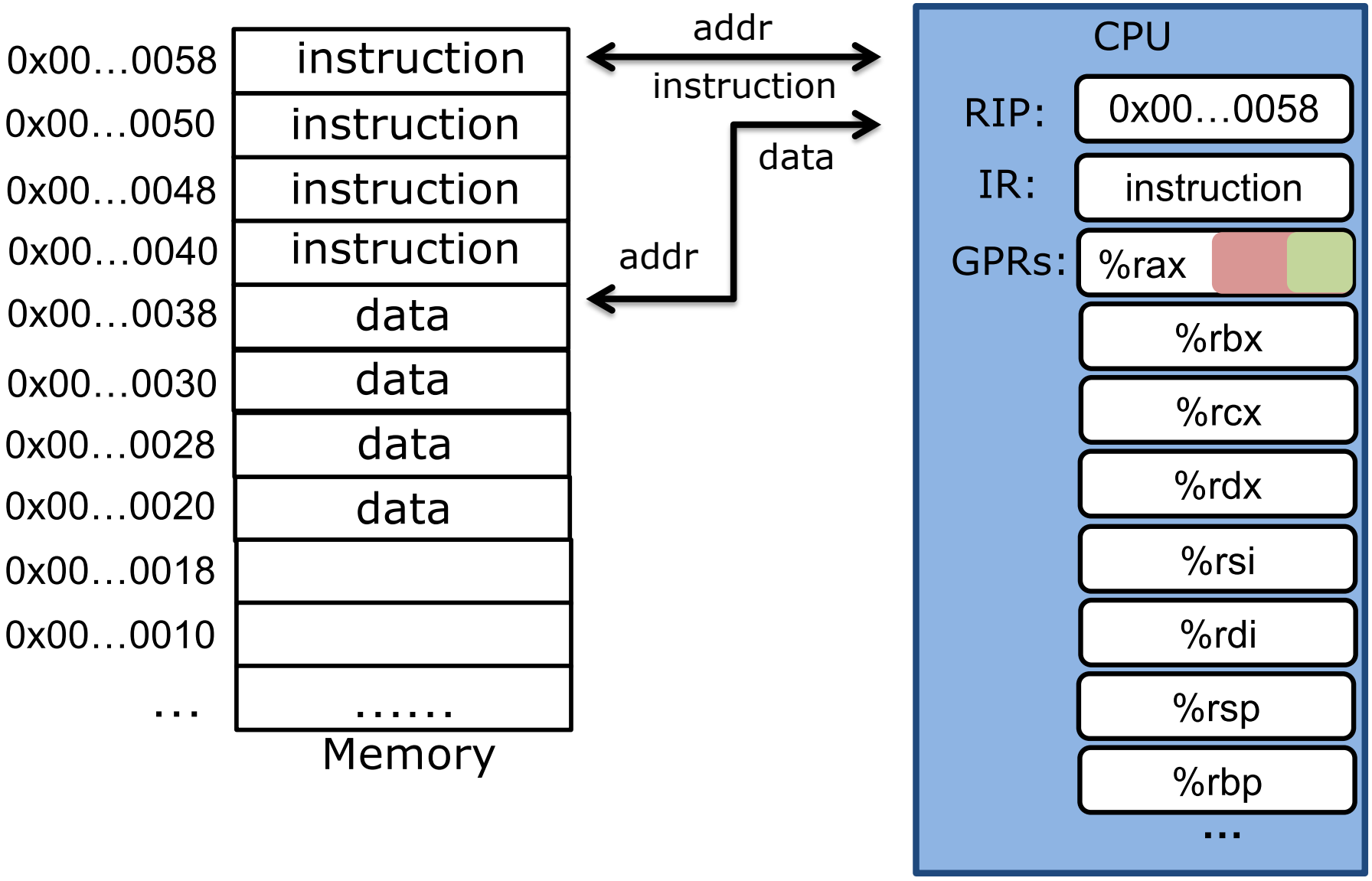
2-byte registers refer to the lower-order 2-bytes of original registers.



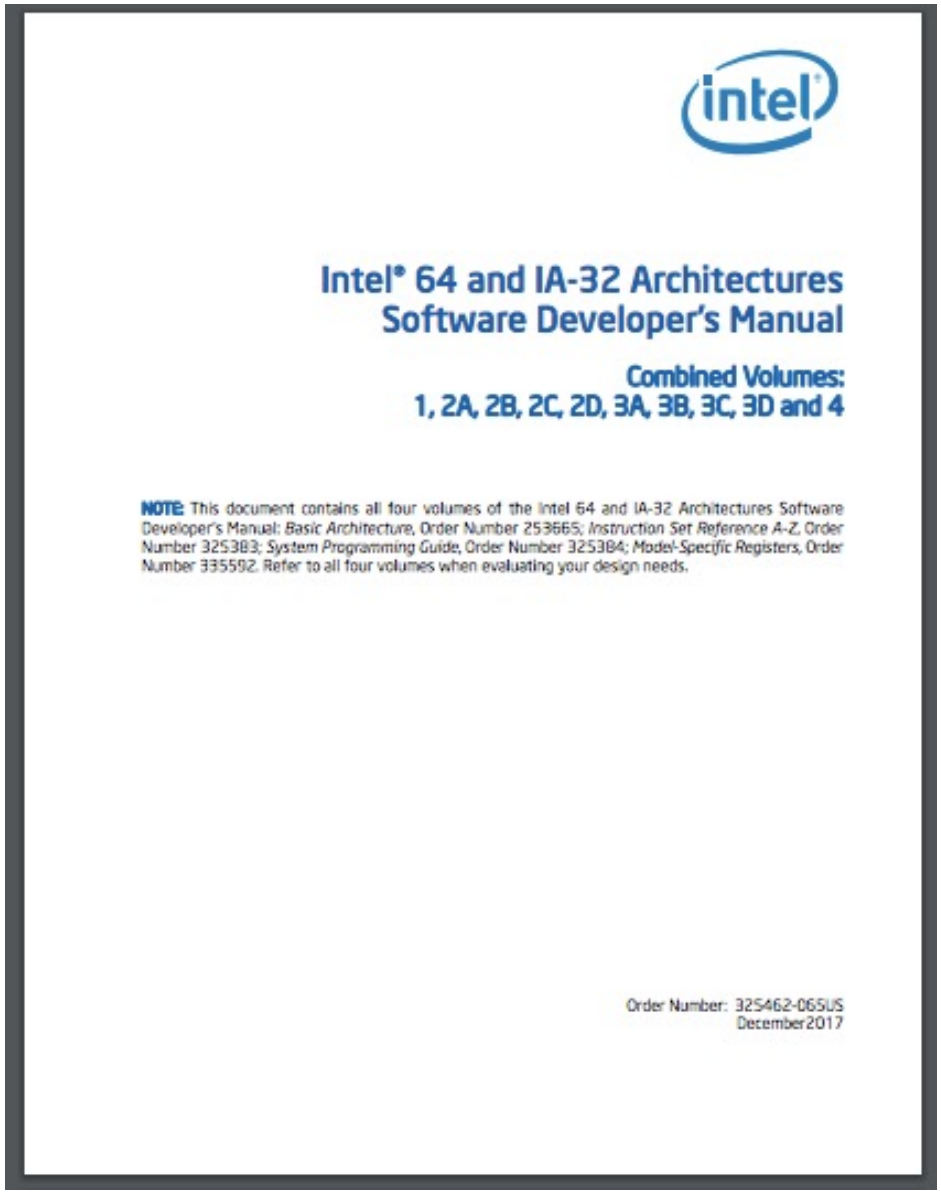
X86-64 general purpose registers: 1-byte



x86-64 execution



X86 ISA



A must-read for compiler and OS writers

https://software.intel.com/en-us/articles/intel-sdm#combined

x86 instruction: Moving data

movq *Source*, *Dest*

- Copy a quadword (64-bit) from the source operand (first operand) to the destination operand (second operand).

We use AT&T (instead of Intel) syntax for assembly

Moving data

suffix

movq *Source*, *Dest*

– Copy a quadword (8-bytes) from the source operand to the destination operand.

Suffix	Name	Size (byte)
b	Byte	1
w	Word	2
l	Long	4
q	Quadword	8

Why using a size suffix?

movq *Source*, *Dest*

- Support **full backward compatibility**
  - New processor can run the same binary file compiled for older processors
- In the Intel x86 world, a word = 16 bits.
  - 8086 refers to 16 bits as a word

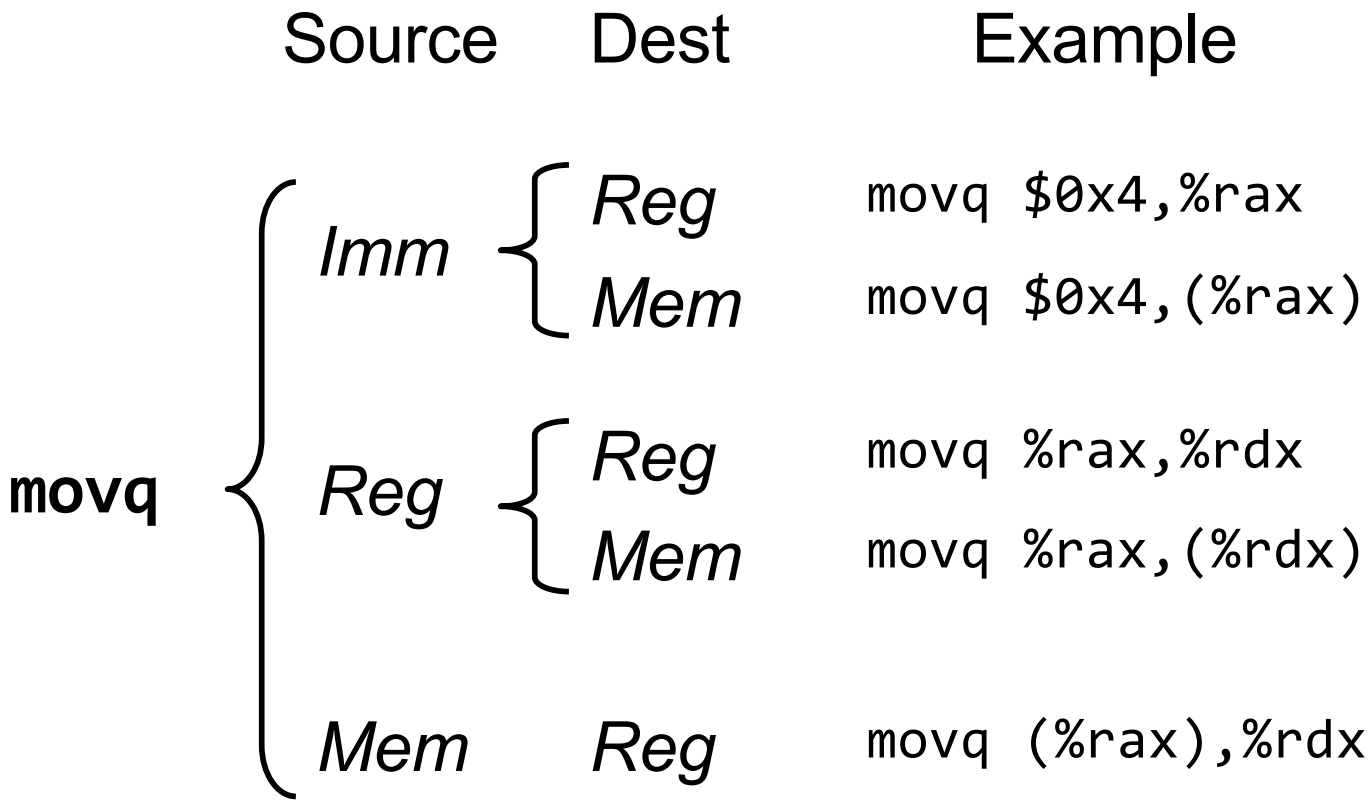
Moving data

movq *Source*, *Dest*

Operand Types

- ***Immediate:*** Constant integer data
  - Prefixed with $
  - E.g: **`$0x400`**, **`$-533`**
- ***Register:*** One of general purpose registers
  - E.g: **`%rax`**, **`%rsi`**
- ***Memory:*** 8 consecutive bytes of memory
  - Indexed by register with various “address modes”
  - Simplest example: **`(%rax)`**

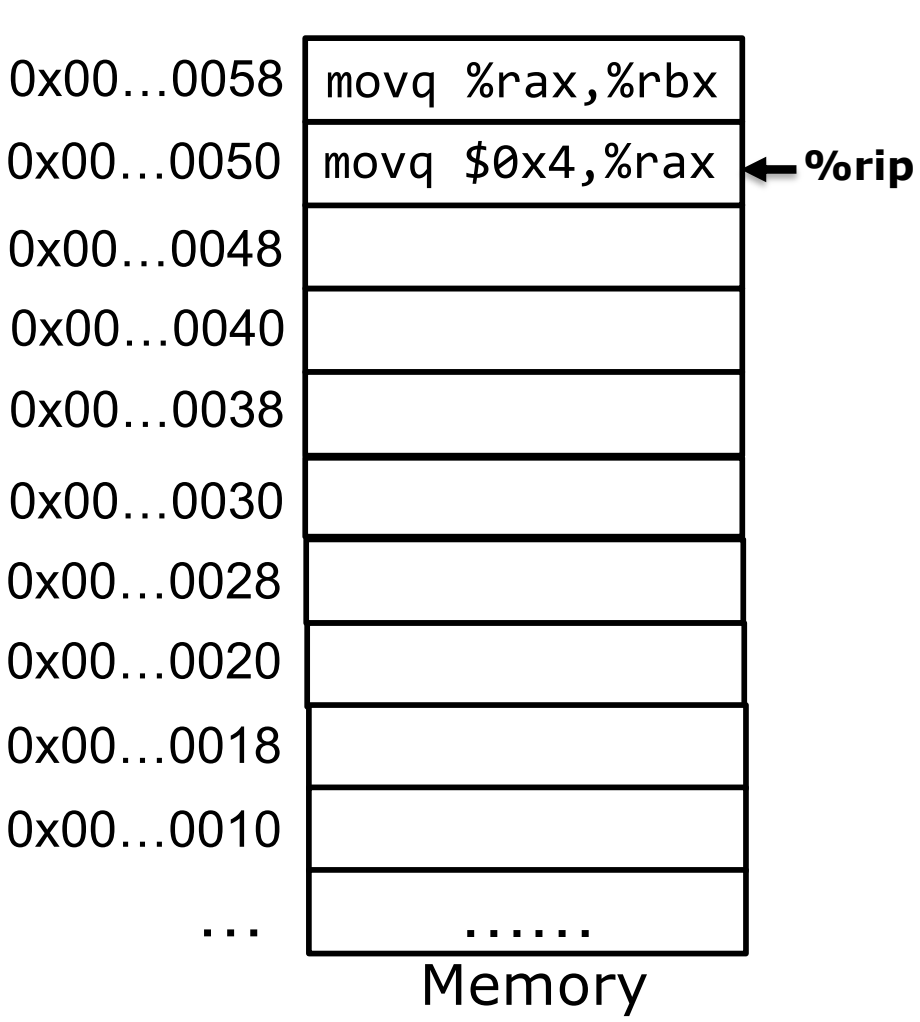
movq Operand combinations

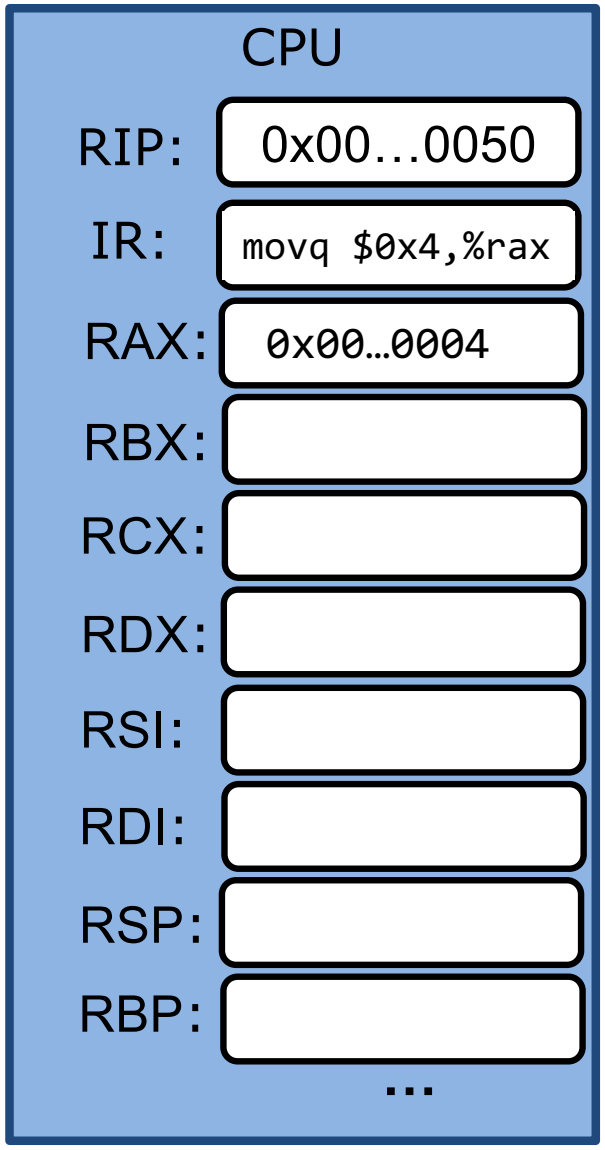


1. Immediate can only be *source*

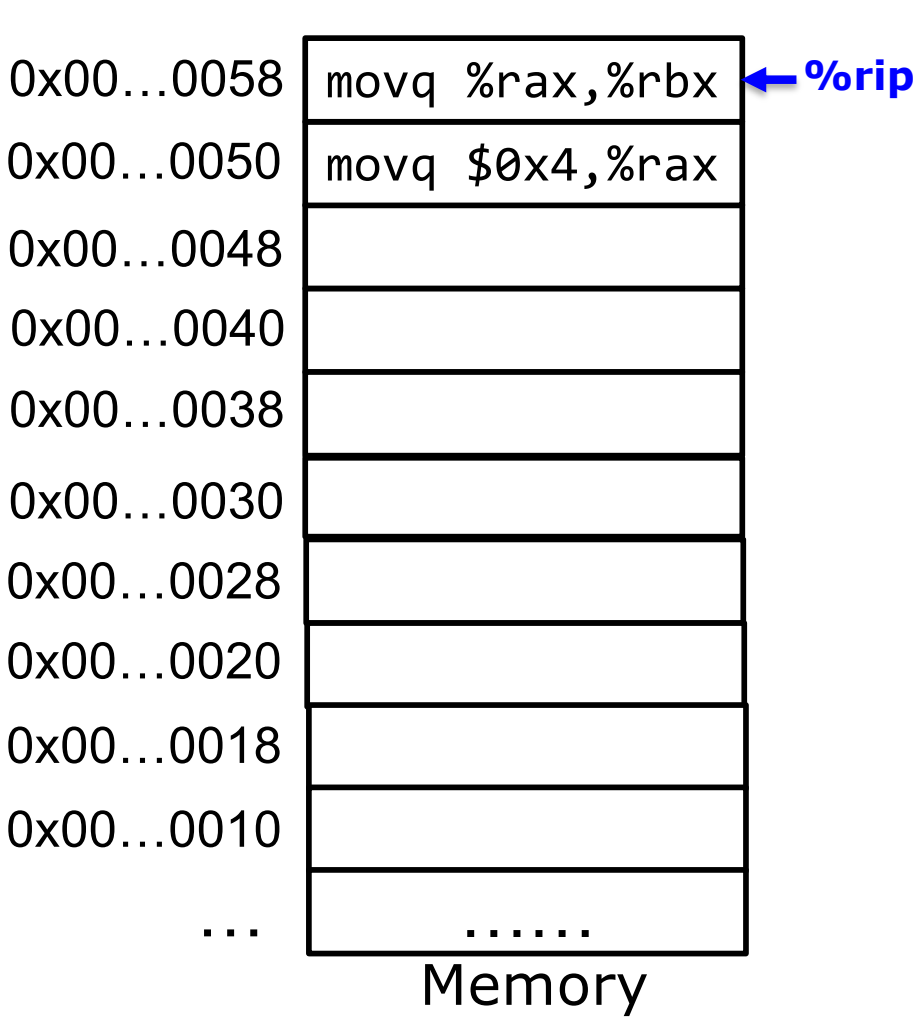
2. No memory-memory mov

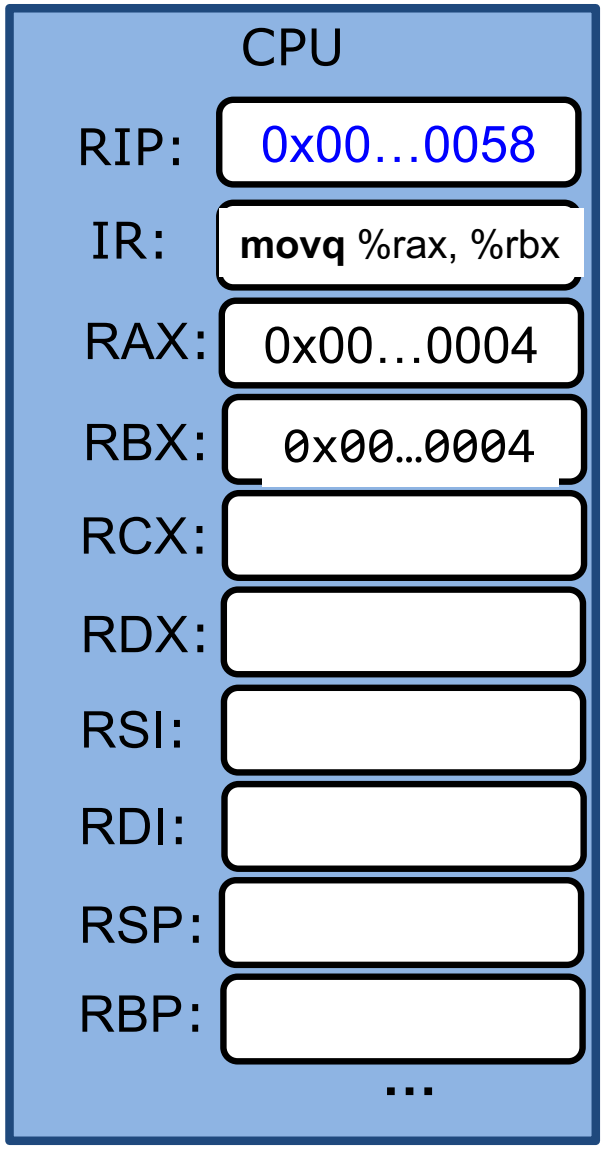
movq *Imm, Reg*


0x00…0058
movq %rax,%rbx
0x00…0050
movq $0x4,%rax
%rip
0x00…0048
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0010
…
……
Memory


CPU
RIP:
0x00…0050
IR:
movq $0x4,%rax
RAX:
0x00…0004
RBX:
RCX:
RDX:
RSI:
RDI:
RSP:
RBP:
…

movq *Reg, Reg*


0x00…0058
movq %rax,%rbx
%rip
0x00…0050
movq $0x4,%rax
0x00…0048
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x00…0010
…
……
Memory


CPU
RIP:
0x00…0058
IR:
movq %rax, %rbx
RAX:
0x00…0004
RBX:
0x00…0004
RCX:
RDX:
RSI:
RDI:
RSP:
RBP:
…

movq *Mem, Reg*

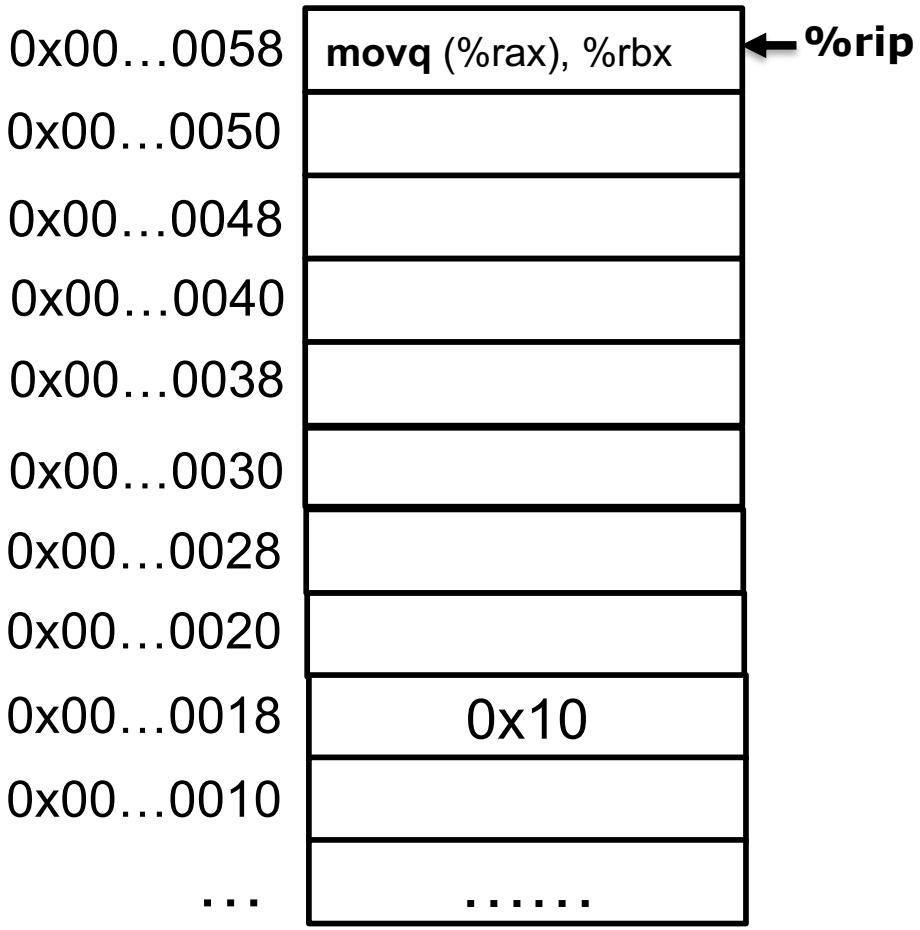
How to represent a “memory” operand?

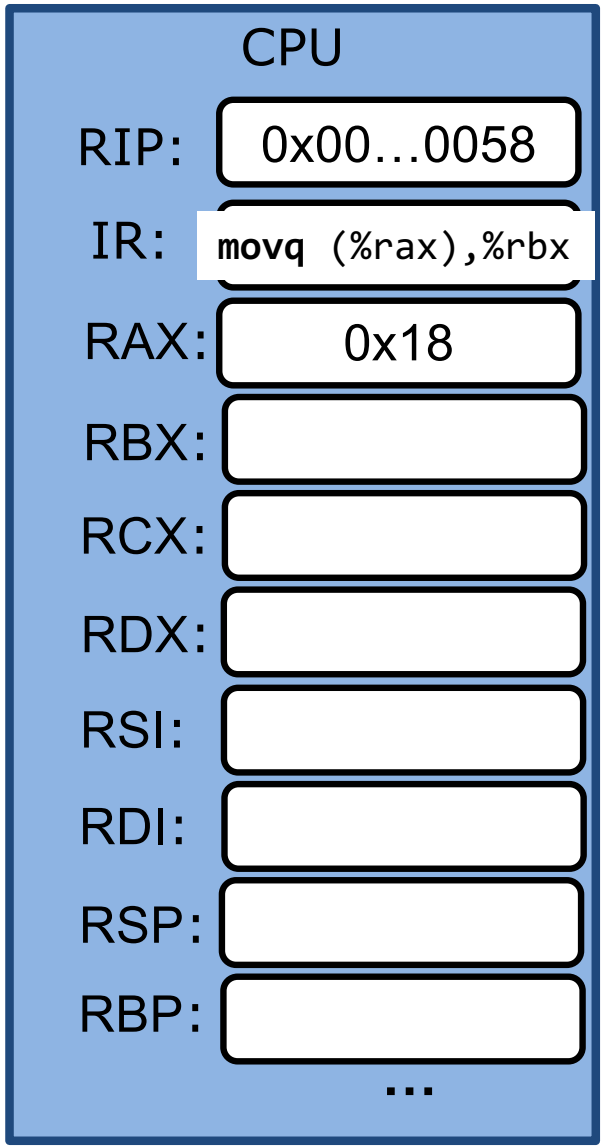
Direct addressing: use register to index memory

(Register)

- The content of the register specifies memory address
- movq (%rax), %rbx

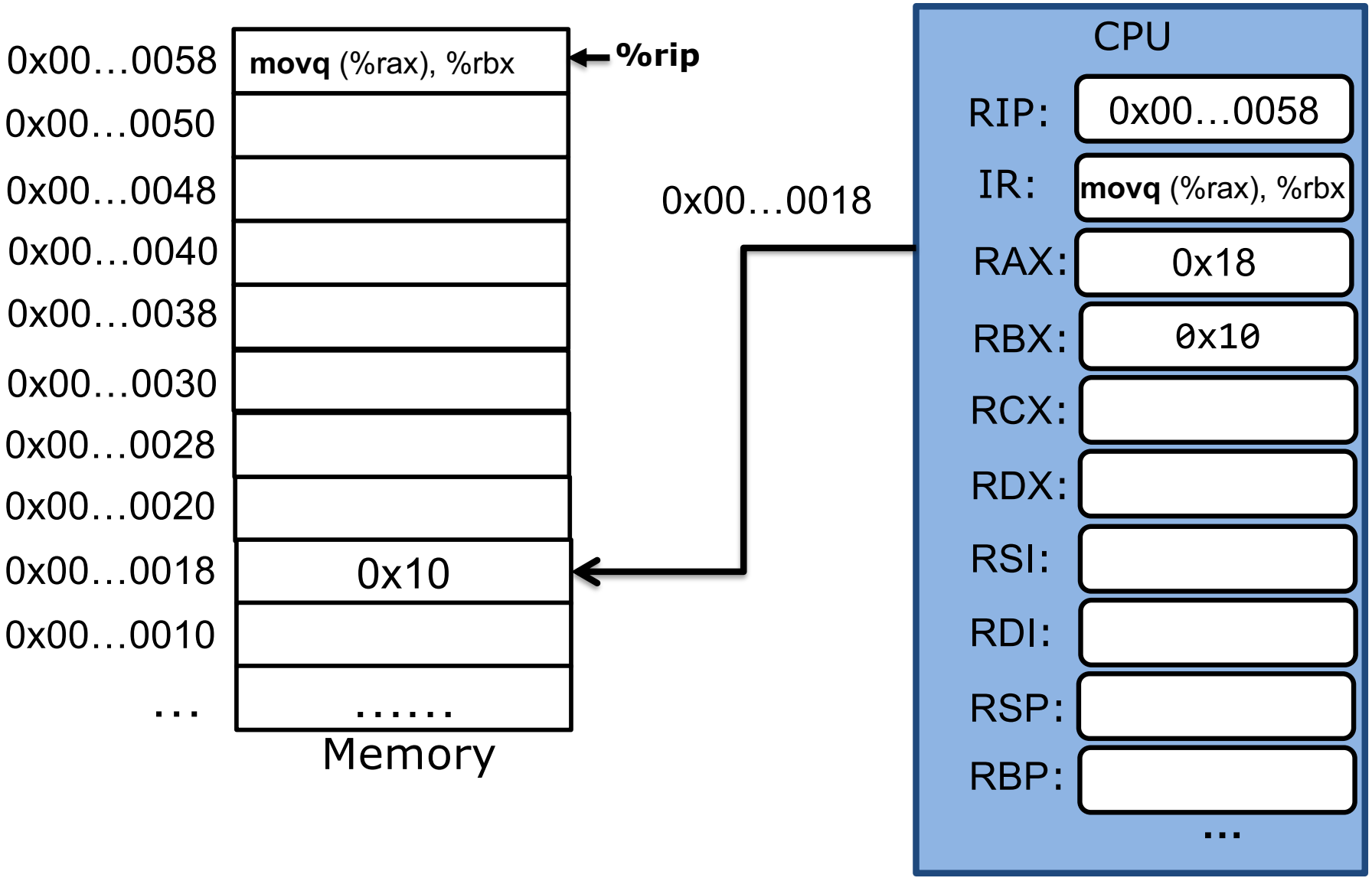
movq *(%rax), %rbx*





How many bytes are copied? Source? Destination?

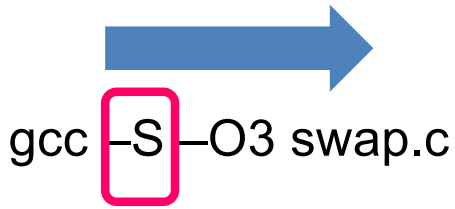
movq *(%rax), %rbx*


0x00…0058
movq (%rax), %rbx
%rip
0x00…0050
0x00…0048
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
0x00…0018
0x10
0x00…0010
…
……
Memory
0x00…0018
CPU
RIP:
0x00…0058
IR:
movq (%rax), %rbx
RAX:
0x18
RBX:
0x10
RCX:
RDX:
RSI:
RDI:
RSP:
RBP:
…

swap function

```
void
swap(long *a, long* b) {

    long tmp = *a;
    *a = *b;
    *b = tmp;

}
```

swap:



Makes gcc output assembly
(human readable machine instructions)

swap function

```
void
swap(long *a, long* b) {

    long tmp = *a;
    *a = *b;
    *b = tmp;

}
```

gcc –S –O3 swap.c

```
swap:
    movq    (%rdi), %rax
    movq    (%rsi), %rdx
    movq    %rdx, (%rdi)
    movq    %rax, (%rsi)
```

%rdi stores a

%rsi stores b

%rax is local variable tmp

swap function

```
void
swap(long *a, long* b) {

    long tmp = *a;
    *a = *b;
    *b = tmp;

}
```

gcc –S –O3 swap.c

```
swap:
        movq    (%rdi), %rax
        movq    (%rsi), %rdx
        movq    %rdx, (%rdi)
        movq    %rax, (%rsi)
```

Use two instructions and %rdx to perform memory to memory move

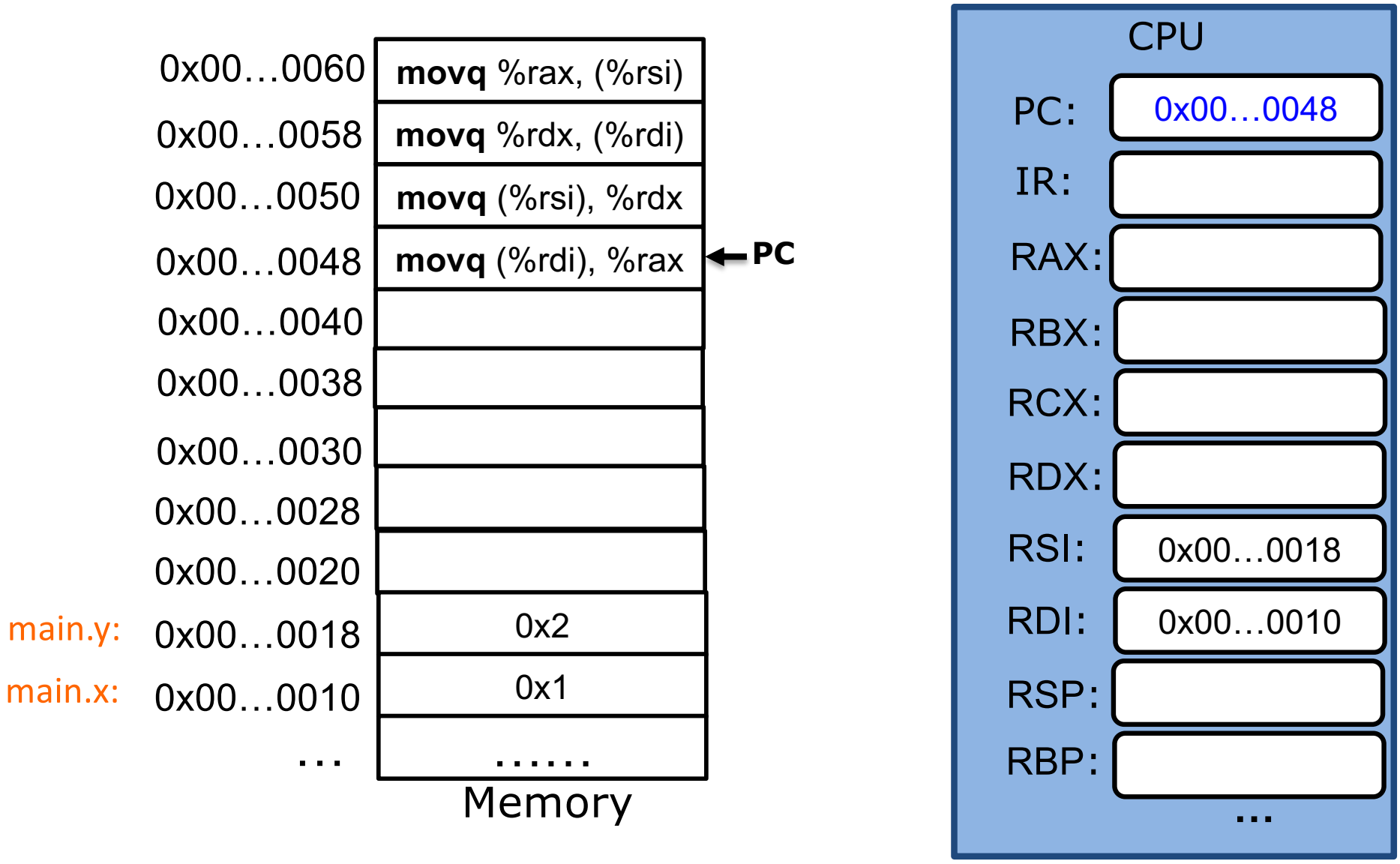
swap function

```
void
swap(long *a, long* b) {

    long tmp = *a;
    *a = *b;
    *b = tmp;

}
```

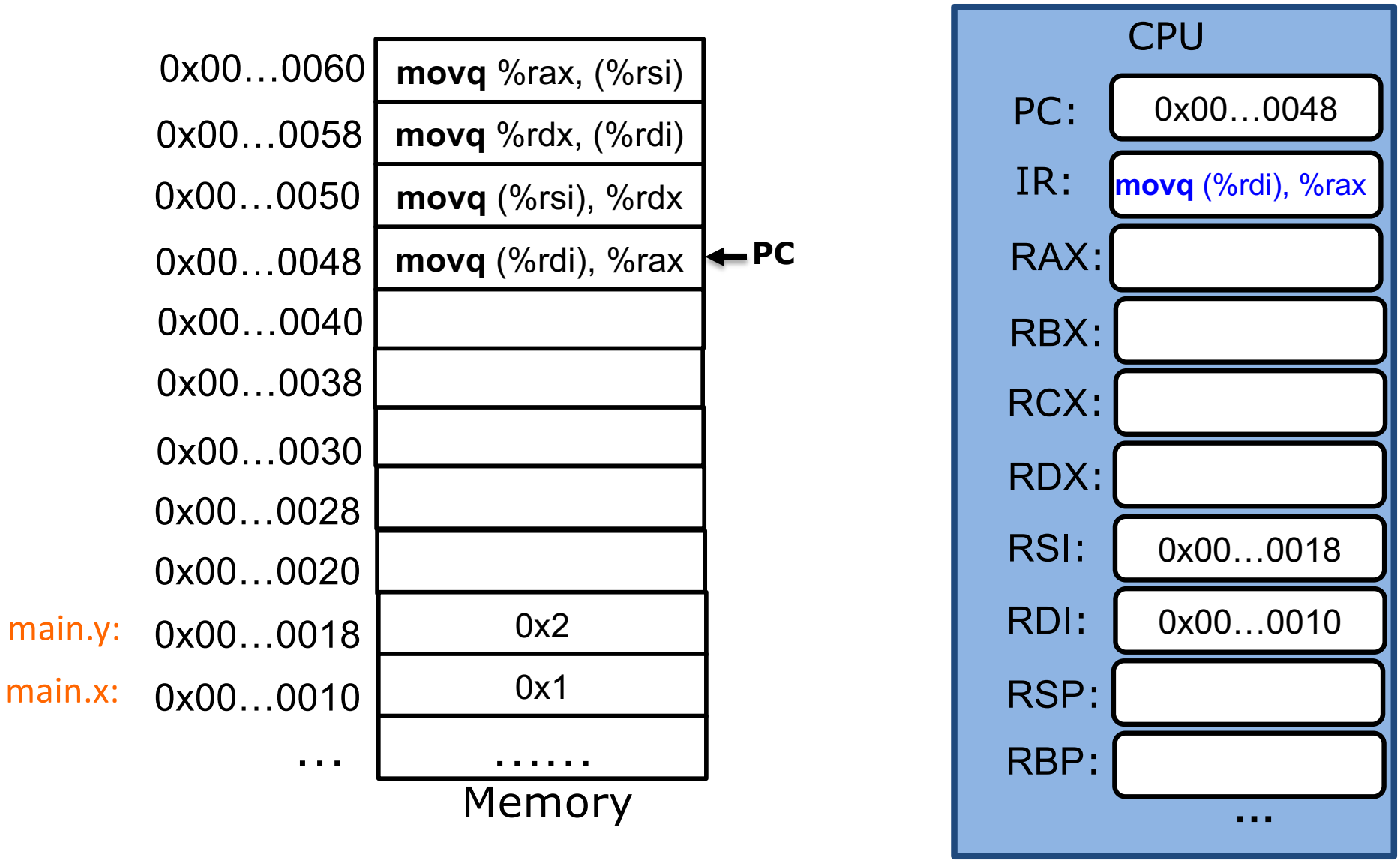
gcc –S –O3 swap.c

```
swap:
        movq    (%rdi), %rax
        movq    (%rsi), %rdx
        movq    %rdx, (%rdi)
        movq    %rax, (%rsi)
```

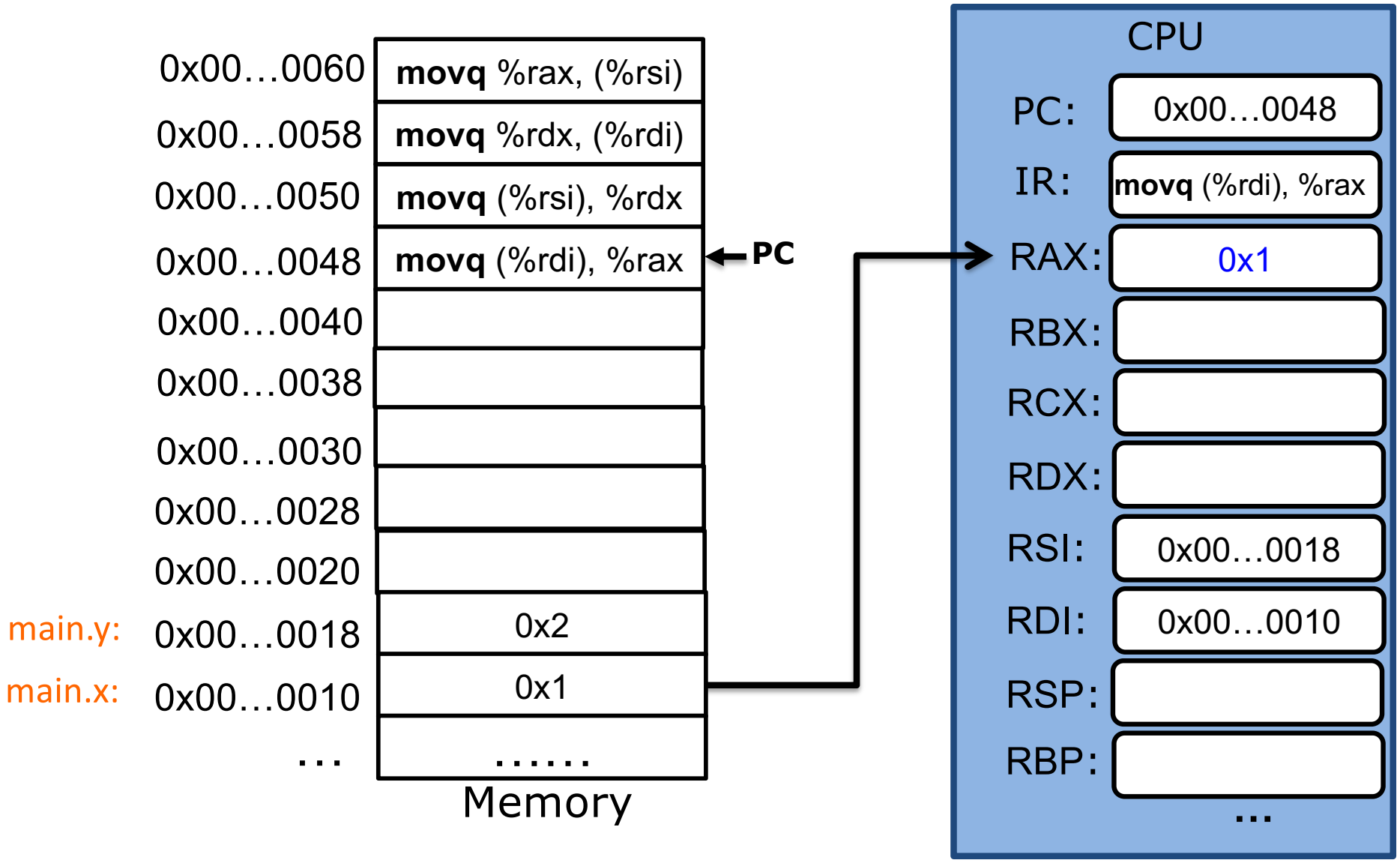
swap func


0x00…0060
movq %rax, (%rsi)
0x00…0058
movq %rdx, (%rdi)
0x00…0050
movq (%rsi), %rdx
0x00…0048
movq (%rdi), %rax
PC
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x1
…
……
Memory
CPU
PC:
0x00…0048
IR:
RAX:
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

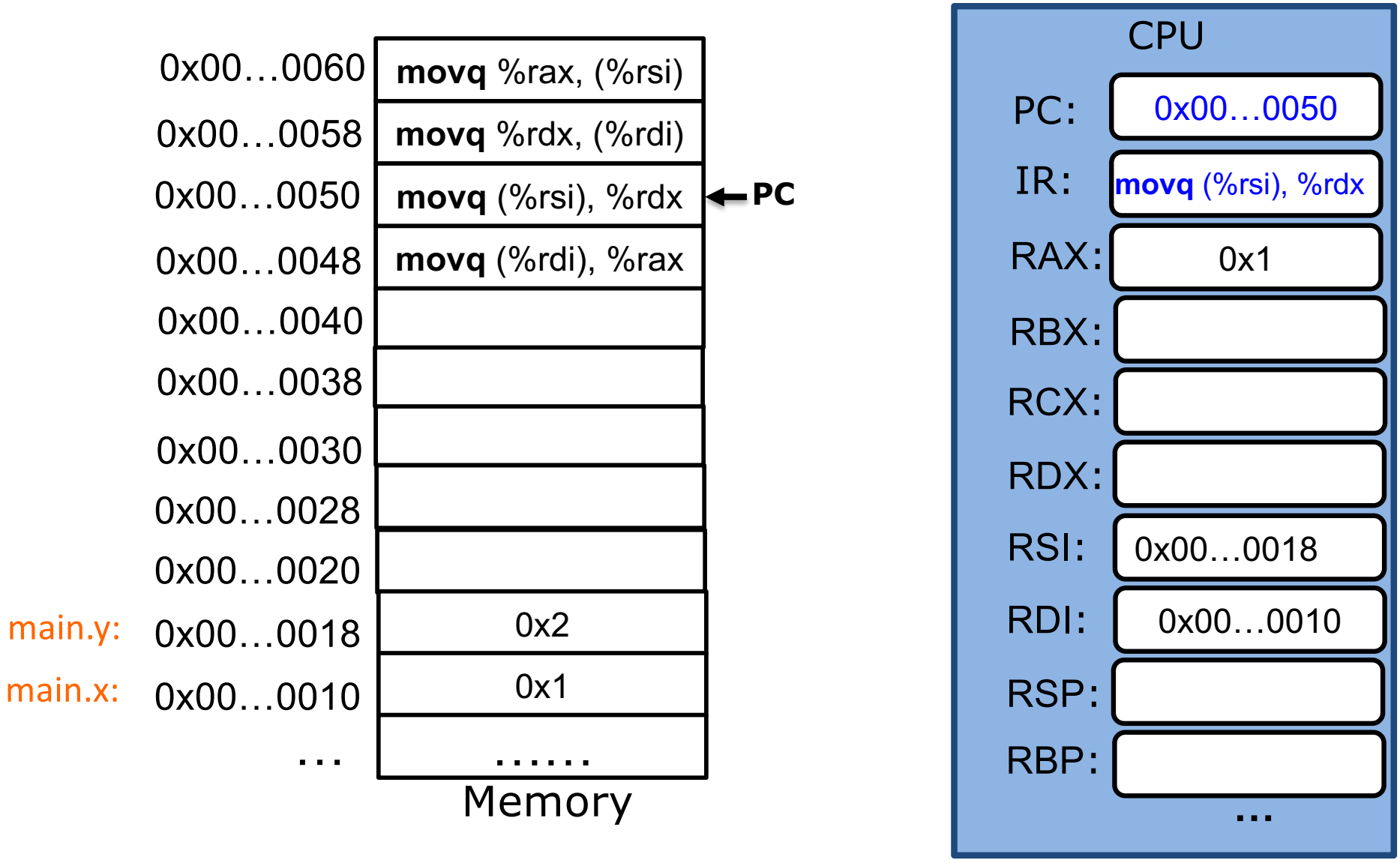
swap func


0x00…0060
movq %rax, (%rsi)
0x00…0058
movq %rdx, (%rdi)
0x00…0050
movq (%rsi), %rdx
0x00…0048
movq (%rdi), %rax
PC
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x1
…
……
Memory
CPU
PC:
0x00…0048
IR:
movq (%rdi), %rax
RAX:
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

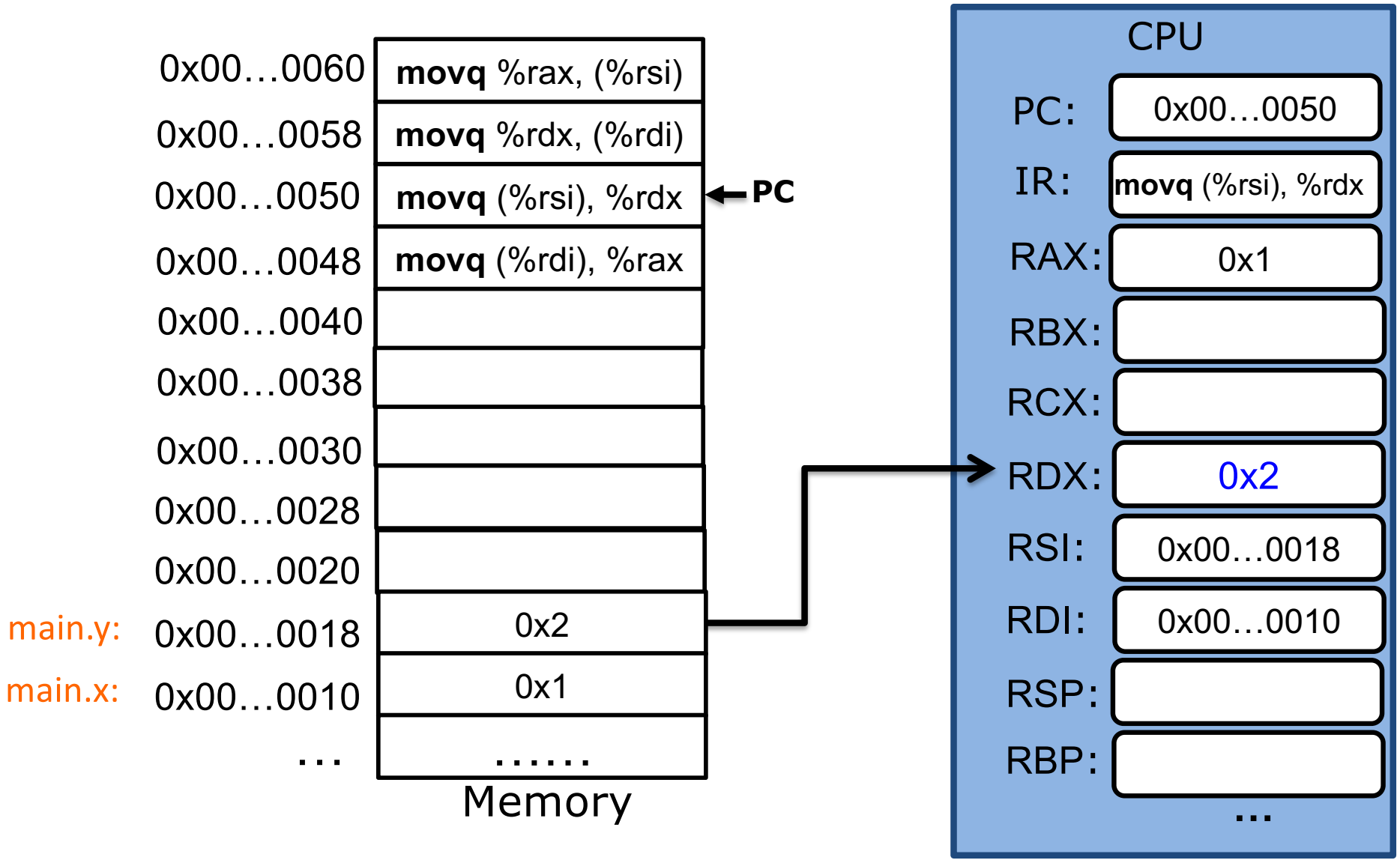
swap func


0x00…0060
movq %rax, (%rsi)
0x00…0058
movq %rdx, (%rdi)
0x00…0050
movq (%rsi), %rdx
0x00…0048
movq (%rdi), %rax
PC
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x1
…
……
Memory
CPU
PC:
0x00…0048
IR:
movq (%rdi), %rax
RAX:
0x1
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

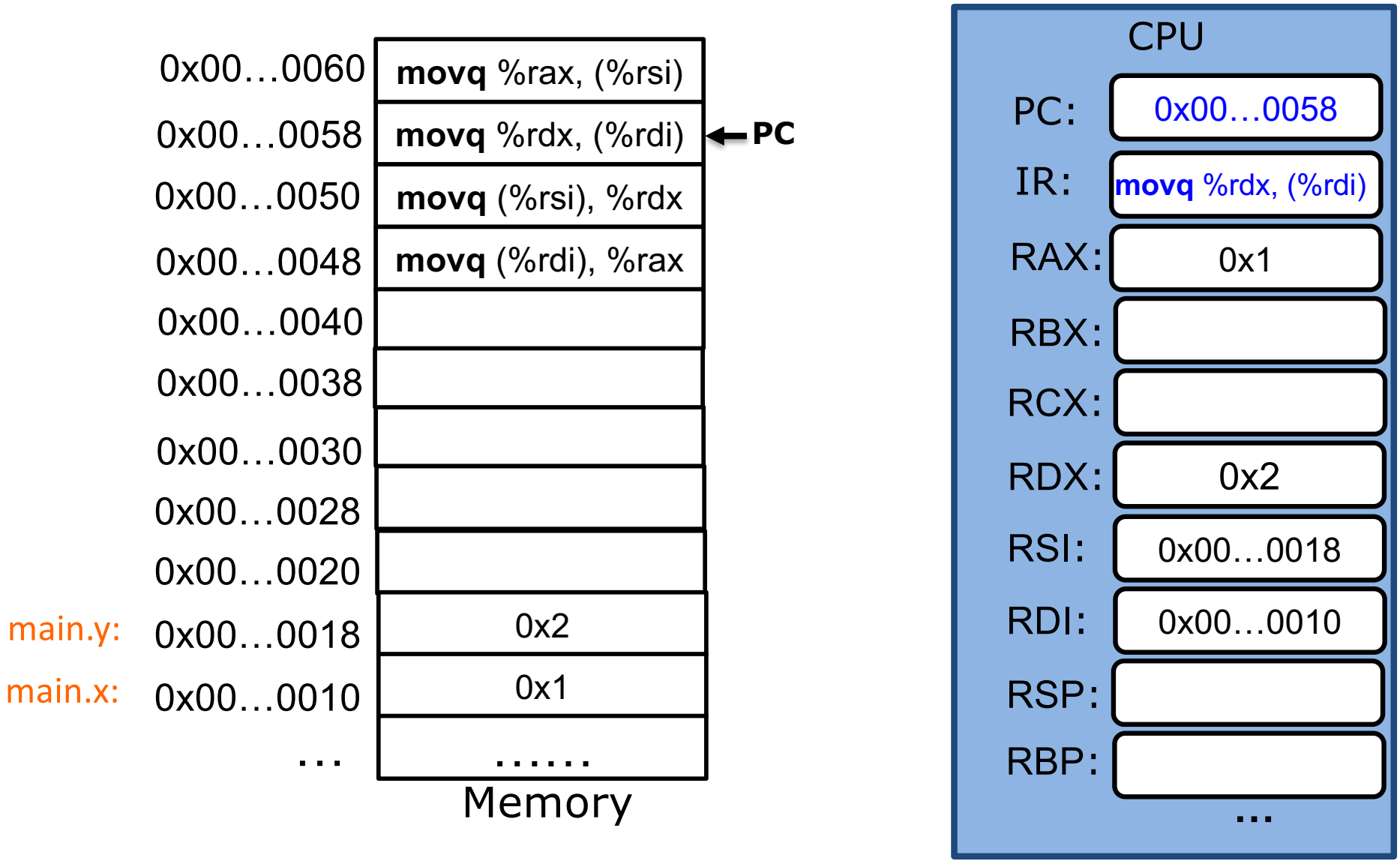
swap func


0x00…0060
movq %rax, (%rsi)
0x00…0058
movq %rdx, (%rdi)
0x00…0050
movq (%rsi), %rdx
PC
0x00…0048
movq (%rdi), %rax
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x1
…
……
Memory
CPU
PC:
0x00…0050
IR:
movq (%rsi), %rdx
RAX:
0x1
RBX:
RCX:
RDX:
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

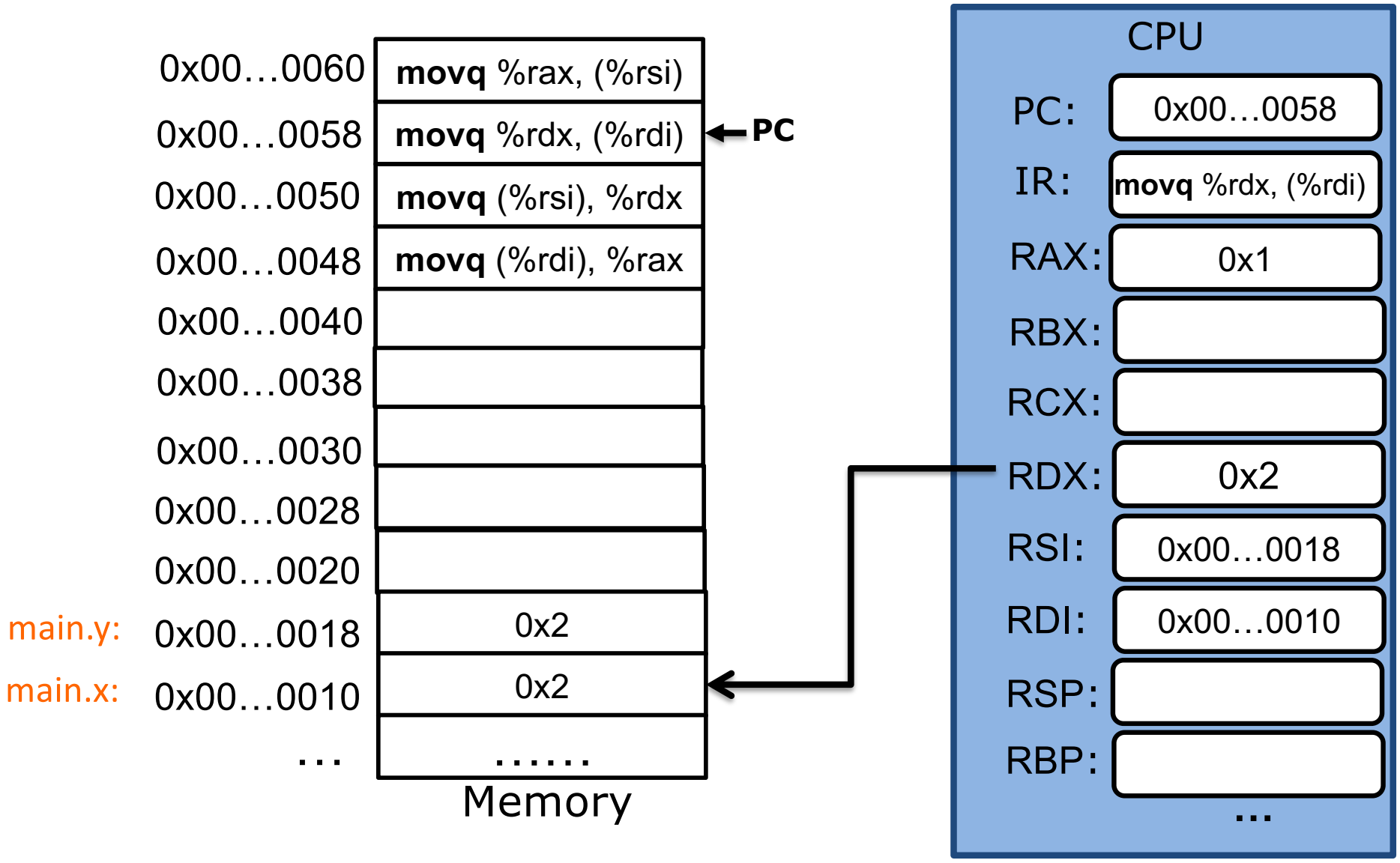
swap func


0x00…0060
movq %rax, (%rsi)
0x00…0058
movq %rdx, (%rdi)
0x00…0050
movq (%rsi), %rdx
PC
0x00…0048
movq (%rdi), %rax
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x1
…
……
Memory
CPU
PC:
0x00…0050
IR:
movq (%rsi), %rdx
RAX:
0x1
RBX:
RCX:
RDX:
0x2
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

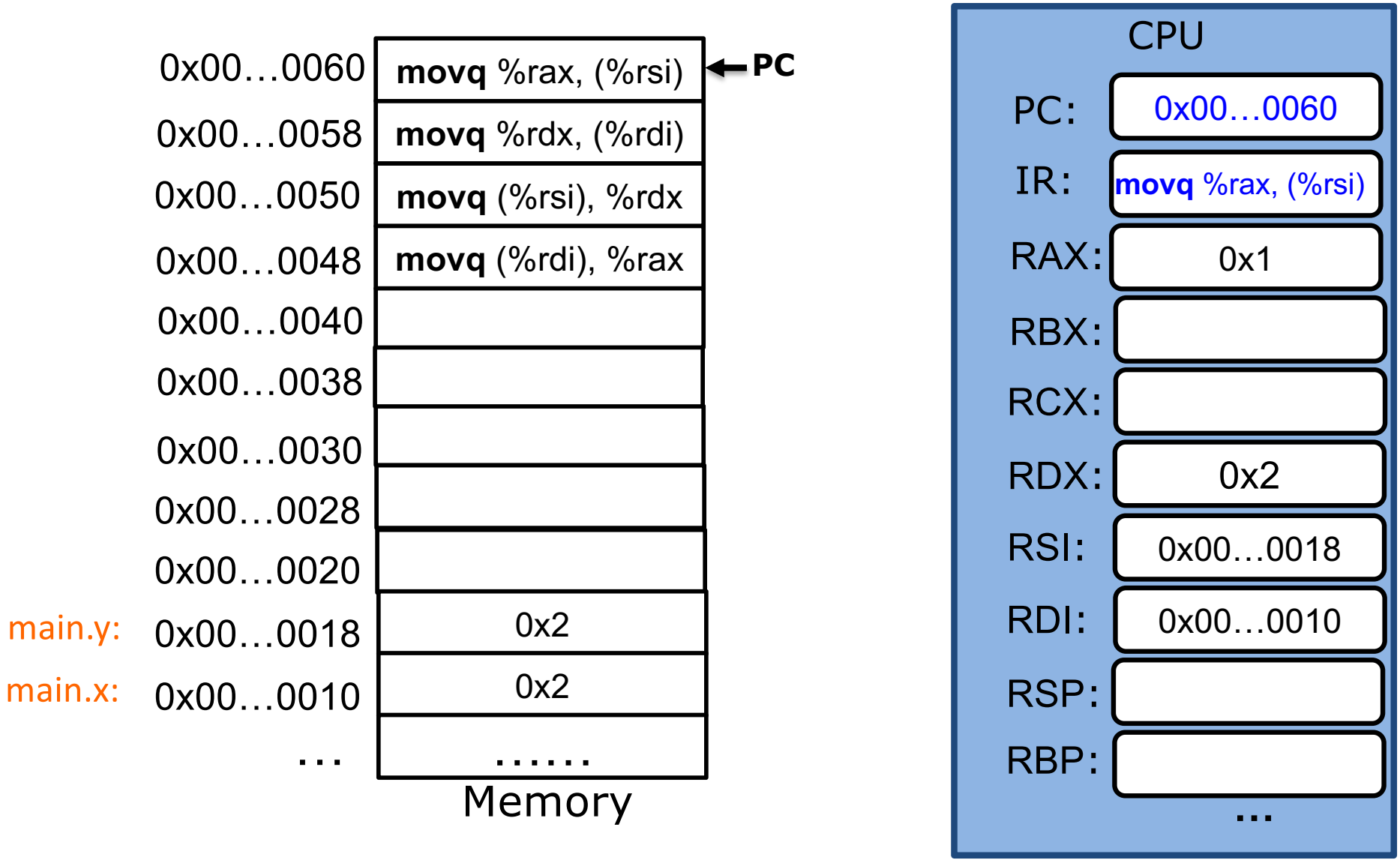
swap func


0x00…0060
movq %rax, (%rsi)
0x00…0058
movq %rdx, (%rdi)
PC
0x00…0050
movq (%rsi), %rdx
0x00…0048
movq (%rdi), %rax
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x1
…
……
Memory
CPU
PC:
0x00…0058
IR:
movq %rdx, (%rdi)
RAX:
0x1
RBX:
RCX:
RDX:
0x2
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

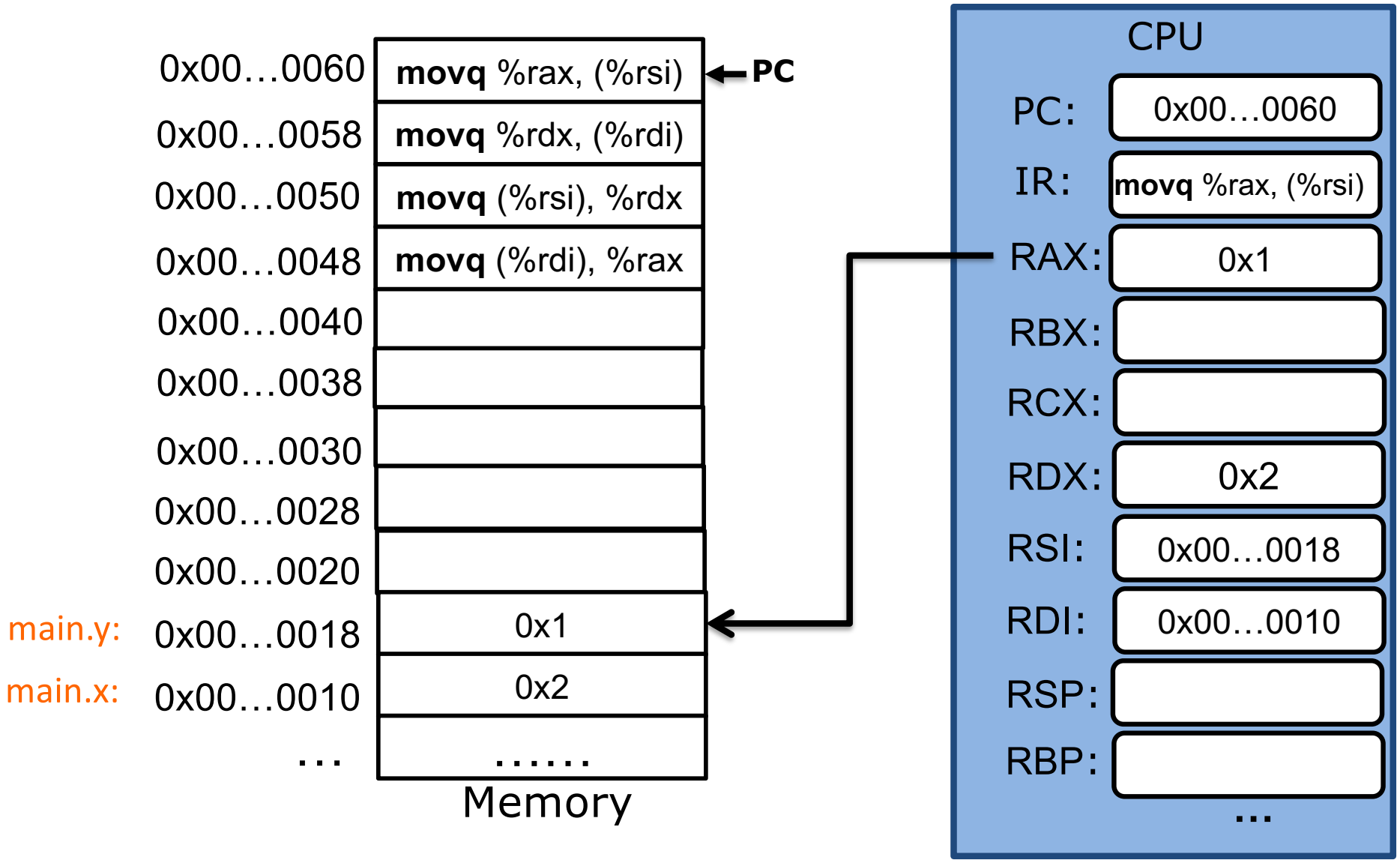
swap func


0x00…0060
movq %rax, (%rsi)
0x00…0058
movq %rdx, (%rdi)
PC
0x00…0050
movq (%rsi), %rdx
0x00…0048
movq (%rdi), %rax
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x2
…
……
Memory
CPU
PC:
0x00…0058
IR:
movq %rdx, (%rdi)
RAX:
0x1
RBX:
RCX:
RDX:
0x2
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

swap func


0x00…0060
movq %rax, (%rsi)
PC
0x00…0058
movq %rdx, (%rdi)
0x00…0050
movq (%rsi), %rdx
0x00…0048
movq (%rdi), %rax
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x2
main.x:
0x00…0010
0x2
…
……
Memory
CPU
PC:
0x00…0060
IR:
movq %rax, (%rsi)
RAX:
0x1
RBX:
RCX:
RDX:
0x2
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

swap func


0x00…0060
movq %rax, (%rsi)
PC
0x00…0058
movq %rdx, (%rdi)
0x00…0050
movq (%rsi), %rdx
0x00…0048
movq (%rdi), %rax
0x00…0040
0x00…0038
0x00…0030
0x00…0028
0x00…0020
main.y:
0x00…0018
0x1
main.x:
0x00…0010
0x2
…
……
Memory
CPU
PC:
0x00…0060
IR:
movq %rax, (%rsi)
RAX:
0x1
RBX:
RCX:
RDX:
0x2
RSI:
0x00…0018
RDI:
0x00…0010
RSP:
RBP:
…

Summary

- Basic hardware execution
  - Instructions and data stored in memory
  - CPU fetches instructions one at a time according to PC
- X86-64 ISA
  - %rip (PC), 16 general-purpose registers
  - movq allows copying data across registers or memory ↔register.